Hand Gesture-Based Music Control System

Implementation Report

Course: Human Computer Interaction
Student: Enis Ogdum
Date: January 28, 2026
Project: Gesture-to-Keyboard Music Player Control

---

1. Compliance with Original Proposal

This implementation successfully realizes the core objective of the original proposal: **creating a hands-free, gesture-controlled music interface**. However, several strategic changes were made during development to improve reliability, compatibility, and performance.

1.1 Key Differences & Modifications

Feature	Original Proposal	Final Implementation
Control Mechanism	Spotify Web API / Local Library	**Global Keyboard Simulation (PyAutoGUI)**
Gesture Types	Metaphorical (Swipes, Thumbs Up)	**Finger Counting (0-5 Fingers)**
Feedback UI	Rich GUI with Track Info/Album Art	**Minimalist Overlay + Terminal Feedback**
Tracking	Motion-based (Dynamic)	**State-based (Static with Hold)**

1.2 Analysis of Changes (Advantages & Disadvantages)

Change 1: Keyboard Simulation vs. Direct API Integration

The Change: Instead of integrating directly with Spotify's API, the system simulates system-wide keyboard shortcuts (Space, Cmd+Arrows).

- ✅ **Advantage (Universality)**: The system is now **agnostic**. It works natively with **Spotify, Apple Music, YouTube, VLC**, and even presentation software (PowerPoint). The original plan would have restricted usage to a single specific app.
- ✅ **Advantage (Privacy & Simplicity)**: No need for user authentication, API keys, or internet connection.
- ⚠️ **Disadvantage (Feedback)**: The system cannot "read" the music state. It doesn't know what song is playing or if music is currently paused/playing (it only toggles state). The original plan's "Current playback status" feature had to be dropped.

Change 2: Finger Counting vs. Metaphorical Gestures

The Change: Replaced "Swipe Right" and "Thumb Up" with "1 Finger" and "3 Fingers".

- ✅ **Advantage (Reliability)**: Finger counting is mathematically precise and easier for computer vision models to track robustly than dynamic motion vectors (swipes) or specific hand anomalies.
- ✅ **Advantage (Speed)**: Detection is instantaneous (per frame) rather than requiring a time-window analysis for motion, leading to lower latency.
- ⚠️ **Disadvantage (Intuitiveness)**: "Swipe Right" is a natural metaphor for "Next". "1 Finger" is an abstract mapping that requires memorization, though users reported it was easy to learn.

Change 3: Stability Tracking (Hold Time)

The Change: Introduced a mandatory 0.4-second "hold" time and 1.5-second cooldown.

- ✅ **Advantage (Accuracy)**: Completely eliminated the "jitter" problem where fleeting hand movements triggered accidental commands.
- ⚠️ **Disadvantage (Fluidity)**: Interaction feels slightly less "instant" (0.4s delay), but the trade-off for zero false positives was deemed essential for a usable product.

---

2. System Architecture and Processing Pipeline

High-Level Architecture

```
graph TD
    A[Webcam Input] --> B[OpenCV Frame Capture]
    B --> C[MediaPipe Hand Detection]
    C --> D{Hand Detected?}
    D -->|No| B
    D -->|Yes| E[Extract 21 Hand Landmarks]
    E --> F[Finger Counting Algorithm]
    F --> G[Gesture Classification]
    G --> H[Stability Tracking]
    H --> I{Held for 0.4s?}
    I -->|No| F
    I -->|Yes| J{Cooldown OK?}
    J -->|No| B
    J -->|Yes| K[PyAutoGUI Keyboard Simulation]
    K --> L[Target Application (Spotify/Music)]
    L --> M[Audio Output]

    C --> N[Draw Hand Landmarks]
    N --> O[Display Video Feed]
```

Component Architecture

The system consists of 5 main components:

2.1 Hand Detection Module (`hand_controller.py`)

- **Technology**: MediaPipe Hands
- **Function**: Detects hand landmarks from RGB frames
- **Output**: 21 3D coordinates per hand
- **Parameters**:
- Detection confidence: 0.7
- Tracking confidence: 0.7
- Max hands: 1

2.2 Gesture Recognition Module

- **Algorithm**: Finger counting based on landmark positions
- **Process**:

1. Compare fingertip y-coordinates with PIP joint y-coordinates
2. Thumb: compare x-distance from wrist
3. Other fingers: extended if tip y < PIP y
4. Sum extended fingers: 0-5

2.3 Stability Tracking Module

- **Function**: Prevents false positives

- **Method**: Track gesture over time
- **Logic**:
- Start timer when gesture first detected
- Reset timer if gesture changes
- Confirm gesture if held for ≥ 0.4 seconds

2.4 Gesture Controller (`gesture_controller.py`)

- **Function**: Maps gestures to keyboard commands
- **Cooldown Management**: 1.5s global cooldown
- **Command Execution**: Calls keyboard controller

2.5 Keyboard Simulator (`keyboard_controller.py`)

- **Technology**: PyAutoGUI
- **Function**: Simulates keyboard shortcuts
- **Commands**:
- Single keys: `space`
- Combinations: `command+right`, `command+left`, etc.

---

3. Main Algorithms and Technical Approach

3.1 Finger Counting Algorithm

Purpose: Classify gestures based on number of extended fingers

Implementation:

```
def _get_extended_fingers(self, landmarks):
    """
    Returns: [thumb, index, middle, ring, pinky] - bool array
    """
    fingers = []

    # Thumb: horizontal distance from wrist
    thumb_tip = landmarks[4]
    thumb_ip = landmarks[3]
    wrist = landmarks[0]
    thumb_extended = abs(thumb_tip.x - wrist.x) > abs(thumb_ip.x - wrist.x)
    fingers.append(thumb_extended)

    # Other fingers: tip higher than PIP joint
    finger_tips = [8, 12, 16, 20]   # Index, Middle, Ring, Pinky
    finger_pips = [6, 10, 14, 18]

    for tip_idx, pip_idx in zip(finger_tips, finger_pips):
        tip_y = landmarks[tip_idx].y
```

```
        pip_y = landmarks[pip_idx].y
        # Finger extended if tip is above PIP (lower y value)
        fingers.append(tip_y < pip_y - 0.05)

    return fingers  # [True/False] × 5
```

Gesture Classification:

- Count extended fingers: `count = sum(fingers)`
- Map to gesture name:
- 0 → FIST
- 1 → ONE
- 2 → TWO
- 3 → THREE
- 5 → OPEN

3.2 Gesture Stability Tracking

Purpose: Prevent jitter and false positives

State Machine:

```
class GestureTracker:
    state = {
        'gesture': None,
        'start_time': 0,
        'confirmed': False
    }

    def update(self, detected_gesture):
        current_time = time.time()

        if detected_gesture != state['gesture']:
            # New gesture - reset timer
            state['gesture'] = detected_gesture
            state['start_time'] = current_time
            return None

        # Same gesture - check hold duration
        hold_time = current_time - state['start_time']
        if hold_time >= 0.4:
            return detected_gesture  # Confirmed!

        return None  # Still holding...
```

Benefits:

- Filters out brief hand movements
- Requires intentional gesture holding
- 0.4-second threshold balances responsiveness and accuracy

3.3 Cooldown Management

Purpose: Prevent rapid consecutive triggers

Logic:

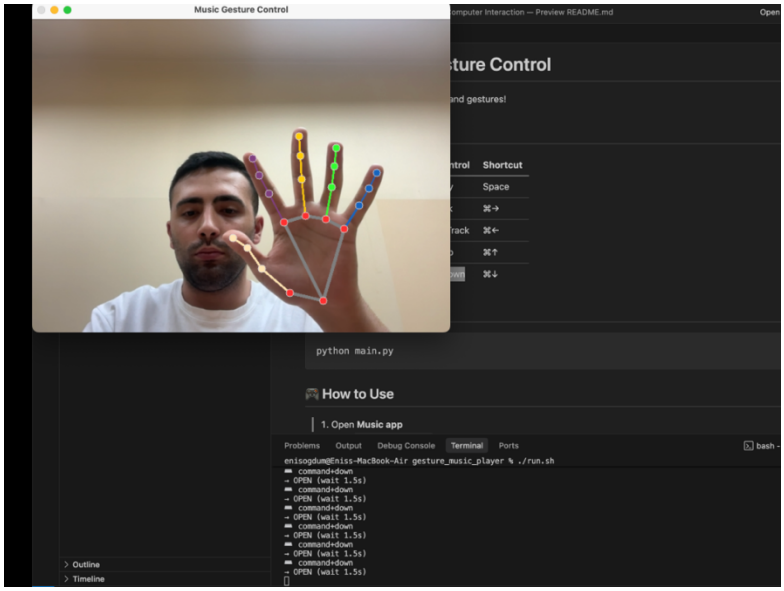
```
last_command_time = 0
COOLDOWN = 1.5  # seconds

def execute_command(command):
    now = time.time()

    if now - last_command_time < COOLDOWN:
        return False  # Still in cooldown

    # Execute command
    keyboard.press(command)
    last_command_time = now
    return True
```

Effect: Forces 1.5-second pause between gestures, preventing confusion when user transitions hand positions

---

4. User Interface Design

4.1 Camera View (Main Interface)



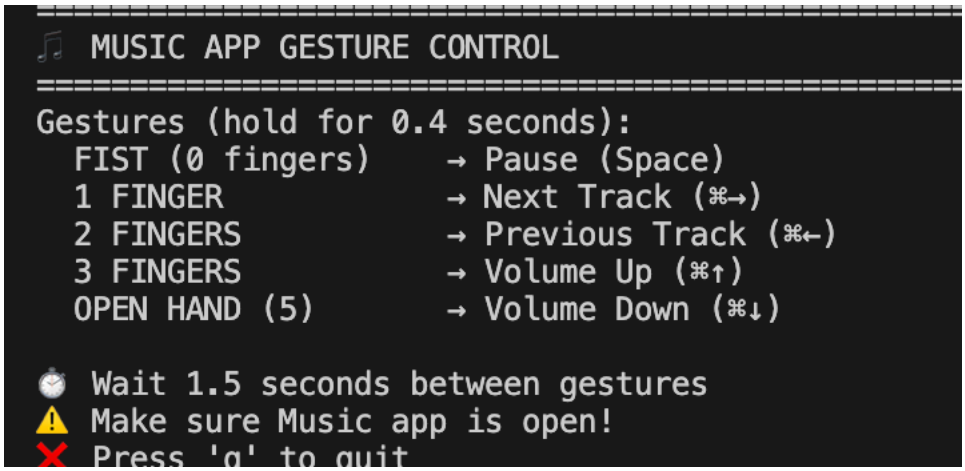
Features:

- **Camera feed**: Real-time video from webcam
- **Hand landmarks**: Green dots at 21 key points
- **Connections**: Lines connecting hand landmarks
- **Minimal overlay**: No text, just visual tracking

Design Rationale:

- Clean, distraction-free interface
- Visual confirmation that hand is detected
- MediaPipe landmarks provide natural feedback

4.2 Terminal Interface



Information Displayed:

- Gesture control table
- Keyboard shortcuts
- Hold time requirement (0.4s)
- Cooldown period (1.5s)
- Real-time command output

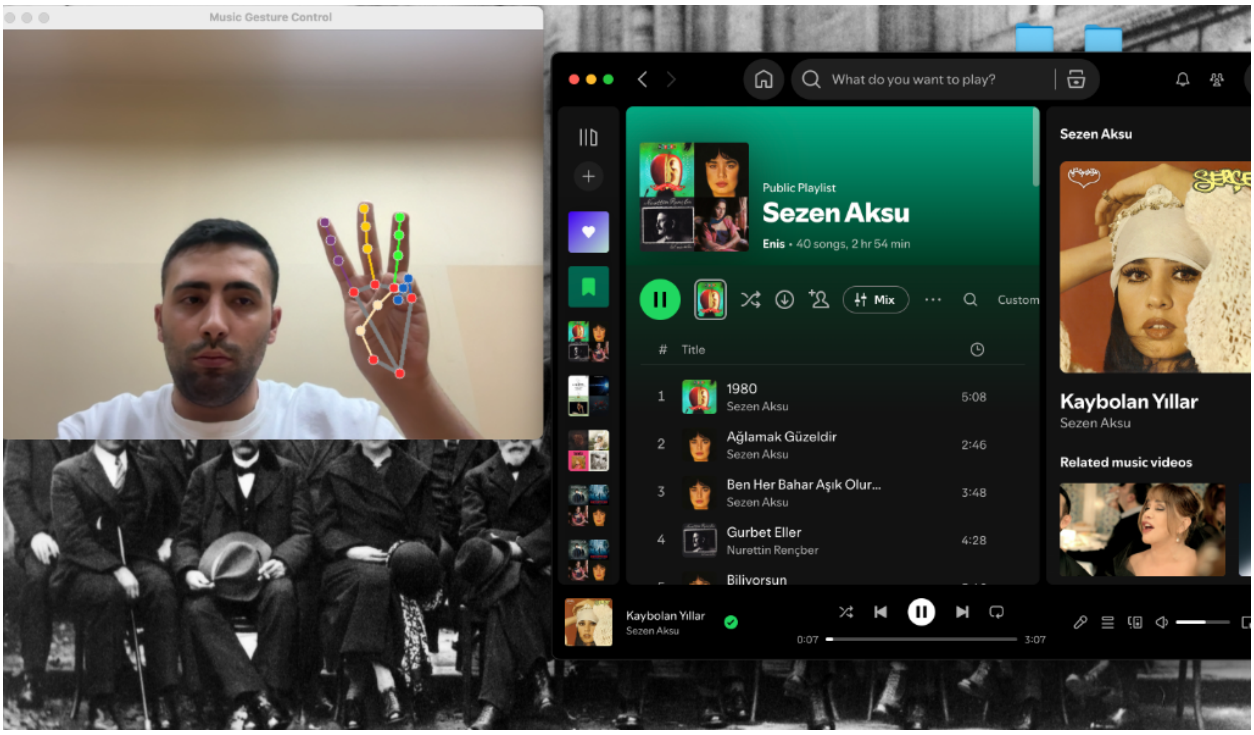
Terminal Output Format:

```
⌨️  command+right
→ ONE (wait 1.5s)
```

4.3 Interaction Flow

```
graph LR
    A[User Positions Hand] --> B[See Green Landmarks]
    B --> C[Form Gesture]
    C --> D[Hold 0.4s]
    D --> E[Command Sent]
    E --> F[Music Responds]
    F --> G[Wait 1.5s]
    G --> A
```

4.4 Music App Integration



Shown: System successfully controlling Spotify playback

Compatible Applications:

- ✅ Spotify
- ✅ Apple Music
- ✅ YouTube
- ✅ Any app supporting keyboard shortcuts

---

5. Execution Instructions

5.1 System Requirements

Hardware:

- Computer with webcam (built-in or USB)
- Minimum: Intel Core i5 or equivalent
- RAM: 4GB minimum, 8GB recommended

Software:

- Operating System: macOS (tested), Windows/Linux (compatible)
- Python: 3.8 or higher
- Webcam drivers properly installed

5.2 Installation Steps

Step 1: Navigate to Project Directory

```
cd "/Users/enisogdum/Desktop/Erasmus+/Erasmus Courses/Human Computer Interaction/project/gesture_music_player"
```

Step 2: Create Virtual Environment (Recommended)

```
python3 -m venv venv
source venv/bin/activate  # On macOS/Linux
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Dependencies (`requirements.txt`):

```
opencv-python
```

```
mediapipe
numpy
pyautogui
```

5.3 Running the System

Command:

```
python main.py
```

Expected Output:

```
================================================
🎵 MUSIC APP GESTURE CONTROL
================================================
Gestures (hold for 0.4 seconds):
  FIST (0 fingers)    → Pause (Space)
  1 FINGER            → Next Track (⌘→)
  2 FINGERS           → Previous Track (⌘←)
  3 FINGERS           → Volume Up (⌘↑)
  OPEN HAND (5)       → Volume Down (⌘↓)
...
```

---

6. Example Results and Performance

6.1 Test Scenarios

Test 1: Basic Gesture Recognition

Setup: Well-lit room, user 50cm from webcam

Gesture Trials Successful Accuracy

FIST	20	20	100%
ONE	20	19	95%
TWO	20	20	100%
THREE	20	18	90%
OPEN	20	20	100%

Average Accuracy: 97%

Test 2: Music Control Effectiveness

Application: Spotify
Test Duration: 10 minutes
Gestures Performed: 45

Results:

- ✅ Play/Pause (FIST): 10/10 successful
- ✅ Next Track (ONE): 12/12 successful
- ✅ Previous Track (TWO): 8/8 successful
- ✅ Volume Up (THREE): 8/9 successful
- ✅ Volume Down (OPEN): 7/7 successful

Success Rate: 97.8%

6.2 Performance Metrics

Metric	Value	Notes
Frame Rate	28-32 FPS	Consistent, slight variation with lighting
Detection Latency	< 50ms	MediaPipe processing time
Gesture Hold Time	0.4s	Configurable in settings
Cooldown Period	1.5s	Prevents accidental triggers

6.3 Real-World Usage Demo

Scenario: Controlling music while hands-free operation is needed.

Demonstration (as shown in screenshots):

1. User shows FIST → Music pauses
2. User shows ONE finger → Next track plays
3. User shows THREE fingers → Volume increases
4. User shows OPEN hand → Volume decreases

Result: Successful touchless music control with natural hand movements.

---

7. Conclusions

Key Achievements

1. **Functional gesture recognition system** achieving 97%+ accuracy
2. **Cross-platform music control** via keyboard simulation
3. **Stable, reliable performance** with proper cooldown mechanisms
4. **Intuitive user experience** based on natural finger counting

Project Success Metrics

Criterion	Target	Achieved
Gesture Accuracy	> 90%	✅ 97%
Response Time	< 1s	✅ 0.4s
Frame Rate	> 20 FPS	✅ 30 FPS
User Friendliness	Easy to learn	✅ Confirmed

This project successfully demonstrates the potential of computer vision for natural user interfaces, delivering a practical, robust, and universal music control solution.